

Recursive Programs

Bird & de Moor, *Algebra of Programming* ch. 6 —
in Freyd diagram order

Conventions

Composition is Juxtaposition in diagram order: $RS =$ “first R , then S ”. Objects α, β, γ ; relations Q, R, S, T, U, V, X, Y ; **maps** (entire and simple) f, g, h, k, m ; coreflexives $C \subset 1$. R° converse, $\subset$ containment, $\cap, \cup$ meet/join, $-$ subtraction, $\emptyset$ empty, Π largest, 1 identity, dom, ran . $[\alpha]$ power object, $\ni : [\alpha] \rightarrow \alpha$ membership, Λ the power transpose ($f = \Lambda R \iff f \ni = R$), $\tau = \Lambda 1$ singleton, $\bigcup = E \ni$ union, $ER = \Lambda(\ni R)$ existential image. F, G relators; $\text{in} : F(\mu F) \rightarrow \mu F$ the initial algebra, $\langle R \rangle$ its catamorphism; $(\mu X : \varphi X)$ the least solution of $X = \varphi X$. Points: $x R y$.

B&dM dictionary. Everything else on the sheet is the book’s, mirrored through this table and nothing more.

B&dM	here
$S \cdot R$	RS composition reverses
id	1
outl, outr	π, π'
inl, inr	ι, ι'
α	in the initial algebra, not an object
$\langle R \rangle$	$\langle R \rangle$
μ	$\bigcup$ (μ is kept for the least fixed point)
$\in, \ni$	$\ni, \ni^\circ$
$T \setminus U, U/T$	$U/T, T \setminus U$ slashes flip, roles kept
$(a, b) \langle R, S \rangle c$	$c \langle R, S \rangle (a, b)$ points reverse too

A row tagged $\checkmark$ has its mirrored statement machine-checked in `AOP/A*.lean`; the rest are hand-mirrored from the page.

6.1 Digits of a number

Convert a positive natural to its decimal digits. The pattern of the whole chapter in one example: specify a function as a refinement of some relation, discover that the relation solves a recursion equation, then read the recursion off as a program.

6.1 $\text{Decimal} ::= \text{wrap } \text{Digit}^+ \mid \text{snoc}(\text{Decimal}, \text{Digit})$, so $([\text{wrap}, \text{snoc}], \text{Decimal})$ is the initial algebra of $FA = \text{Digit}^+ + (A \times \text{Digit})$.

$\text{val} = ([\text{embed}, \text{op}]) : \text{Decimal} \rightarrow \text{Nat}^+$, with embed the inclusion of digits and $\text{op}(n, d) = 10n + d$.

(6.1) $\text{digits} \subset \text{val}^\circ$ — a *functional refinement* of val° , which is not yet known to be a map. This is the specification.

$$\begin{aligned}
 & \text{val}^\circ \\
 = & \text{definition} \\
 & ([\text{embed}, \text{op}])^\circ \\
 = & \text{catamorphisms} \\
 & ([\text{wrap}, \text{snoc}]^\circ (F \text{val}) [\text{embed}, \text{op}])^\circ \\
 = & \text{converse} \\
 & [\text{embed}, \text{op}]^\circ (F \text{val}^\circ) [\text{wrap}, \text{snoc}] \\
 = & \text{definition of } F \\
 & [\text{embed}, \text{op}]^\circ (1 + (\text{val}^\circ \times 1)) [\text{wrap}, \text{snoc}] \\
 = & \text{coproduct} \\
 & [\text{embed}, \text{op}]^\circ [\text{wrap}, (\text{val}^\circ \times 1) \text{snoc}] \\
 = & \text{coproduct} \\
 & (\text{embed}^\circ \text{wrap}) \cup (\text{op}^\circ (\text{val}^\circ \times 1) \text{snoc})
 \end{aligned}$$

6.1 $\checkmark$ So $\text{val}^\circ = (\text{embed}^\circ \text{wrap}) \cup (\text{op}^\circ (\text{val}^\circ \times 1) \text{snoc})$.

$\text{op}(n, d) = m \iff n = m \text{ div } 10 \wedge d = m \text{ mod } 10$, so op° is defined exactly for $m \geq 10$ and embed° exactly for $m < 10$: the two branches have disjoint domains, and the join becomes a conditional.

$\text{val}^\circ m = \text{wrap } m$ if $m < 10$, else $\text{snoc}(\text{val}^\circ(m \text{ div } 10), m \text{ mod } 10)$. The recursion terminates because $m \geq 10 \implies m \text{ div } 10 < m$, so val° is a total function and $\text{digits} = \text{val}^\circ$.

$\text{digits } m = [m]$ if $m < 10$, else $\text{digits}(m \text{ div } 10) \# [m \text{ mod } 10]$ — quadratic, because snoc on lists is linear.

With an accumulator, digits $m = f(m, [])$ where
 $f(m, x) = [m] \# x$ if $m < 10$, else
 $f(m \text{ div } 10, [m \text{ mod } 10] \# x)$ — linear, and correct at
0.

6.2 Least fixed points

T 6.1 ✓ **KNASTER–TARSKI** φ a monotonic mapping on the arrows $\alpha \rightarrow \beta$ of a locally complete allegory. Then $\varphi X \subset X$ and $\varphi X = X$ have the same least solution, written $(\mu X : \varphi X)$; dually $X \subset \varphi X$ and $X = \varphi X$ have the same greatest solution.

Proof: $(\mu X : \varphi X) = \bigcap \{X : \varphi X \subset X\}$. Local completeness is what makes that meet exist.

in is an isomorphism, so $\llbracket R \rrbracket$ is the unique — hence both least and greatest — solution of $X = \text{in}^\circ(FX)R$, and $\varphi X = \text{in}^\circ(FX)R$ is monotonic. Knaster–Tarski then gives:

$$(6.2) \quad \checkmark \quad \llbracket R \rrbracket \subset X \iff \text{in}^\circ(FX)R \subset X.$$

$$(6.3) \quad \checkmark \quad X \subset \llbracket R \rrbracket \iff X \subset \text{in}^\circ(FX)R.$$

$$(6.4) \quad \checkmark \quad \textbf{FUSION}, \subset \llbracket T \rrbracket \subset \llbracket R \rrbracket S \iff (FS)T \subset RS.$$

$$(6.5) \quad \checkmark \quad \textbf{FUSION}, \supset \llbracket R \rrbracket S \subset \llbracket T \rrbracket \iff RS \subset (FS)T.$$

Equality of the two hypotheses gives back the catamorphism fusion law of ch. 3.

Ex 6.3 ✓ **KLEENE** φ continuous — it preserves joins of ascending chains — then $(\mu X : \varphi X) = \bigcup \{\varphi^n \emptyset : 0 \leq n\}$.

Ex 6.4 ✓ **FIXED POINT INDUCTION** $(\mu X : \varphi X) \subset A \iff \varphi A \subset A$; by Kleene, $(\forall X. X \subset A \implies \varphi X \subset A)$ suffices too.

Ex 6.5 The least solution of $\varphi X \subset X$ satisfies $\varphi X = X$ — Lambek’s lemma, for the containment order read as a category and φ as a functor on it.

Ex 6.7 ✓ $\llbracket R \rrbracket \subset \llbracket S \rrbracket \iff (F\llbracket S \rrbracket)R \subset (F\llbracket S \rrbracket)S$. It does *not* need $R \subset S$.

Ex 6.8 ✓ R **DIFUNCTIONAL** $\triangleq R = RR^\circ R$; the difunctional closure of R is a least fixed point.

6.3 Hylomorphisms

6.3 A **HYLOMORPHISM** is a catamorphism after the converse of one, $\llbracket S \rrbracket^\circ \llbracket R \rrbracket$ for $R : F\alpha \rightarrow \alpha$ and $S : F\beta \rightarrow \beta$. The initial type μF is the *intermediate data structure*: it is built by $\llbracket S \rrbracket^\circ$ and consumed by $\llbracket R \rrbracket$, and never appears in the answer.

T 6.2 ✓ **HYLOMORPHISM THEOREM**

$$\llbracket S \rrbracket^\circ \llbracket R \rrbracket = (\mu X : S^\circ(FX)R).$$

S° divides, FX conquers, R combines.

Proof in two halves: $\llbracket S \rrbracket^\circ \llbracket R \rrbracket$ is a fixed point (functors and the two catamorphism laws), and it is below every prefixed point — that half goes through division, $\llbracket S \rrbracket^\circ \llbracket R \rrbracket \subset X \iff \llbracket R \rrbracket \subset X / \llbracket S \rrbracket^\circ$, which is the standard move for least fixed points.

C 6.1 ✓ $FX = GX + HX$, so an F -algebra is a pair:

$$\llbracket S_1, S_2 \rrbracket^\circ \llbracket R_1, R_2 \rrbracket = (\mu X : (S_1^\circ(GX)R_1) \cup (S_2^\circ(HX)R_2)).$$

in is an isomorphism and $\llbracket \text{in} \rrbracket = 1$, so every catamorphism and every converse of one is already a hylomorphism.

Ex 6.10 ✓ Hylomorphisms preserve simplicity: R and S simple $\implies \llbracket S \rrbracket^\circ \llbracket R \rrbracket$ simple.

6.4 Fast exponentiation and modulus

6.4 $\exp a = \langle \text{one}, \text{mult } a \rangle : \text{Nat} \rightarrow \text{Nat}$ encodes $a^0 = 1$ and $a^{b+1} = a \times a^b$ — $O(b)$ steps. Divide and conquer brings it to $O(\log b)$.

$\text{Bin} = \text{list1 Bit}$, $\text{Bit} = \{0, 1\}$; $\text{convert} = \langle \text{zero}, \text{shift} \rangle : \text{Bin} \rightarrow \text{Nat}$ with $\text{shift}(n, d) = 2n + d$ reads a bit sequence as a number. It is simple.

$\exp a$
 $\supseteq$ convert simple, so $\text{convert}^\circ \text{convert} \subset 1$
 $\text{convert}^\circ \text{convert} (\exp a)$
 $=$ fusion, with $\text{op } a(n, d) = (d = 0 \rightarrow n^2, a \times n^2)$
 $\text{convert}^\circ \langle \text{one}, \text{op } a \rangle$
 $=$ Corollary 6.1
 $(\mu X : (\text{zero}^\circ \text{one}) \cup (\text{shift}^\circ (X \times 1)(\text{op } a)))$

6.4 ✓ The fusion step needs $\text{zero} (\exp a) = \text{one}$ and $\text{shift} (\exp a) = (\exp a \times 1)(\text{op } a)$. zero and shift have disjoint ranges, so the join is again a conditional:

$\exp a \ b = 1$ if $b = 0$, else $\text{op } a(\exp a(b \text{ div } 2), b \text{ mod } 2)$.

6.4 ✓ Same idea for $a \text{ mod } b$: $\text{mod } b = \langle \text{zero}, \text{succ } b \rangle$ with $\text{succ } b \ a = (a = b - 1 \rightarrow 0, a + 1)$ costs $O(a)$; fusing through convert gives

$\text{mod } b \supset (\mu X : (\text{zero}^\circ \text{zero}) \cup (\text{shift}^\circ (X \times 1)(\text{op } b)))$

with $\text{op } b(r, d) = (n \geq b \rightarrow n - b, n)$ and $n = 2r + d$.

Conditions: $\text{zero} (\text{mod } b) = \text{zero}$ and $\text{shift} (\text{mod } b) = (\text{mod } b \times 1)(\text{op } b)$. The program is $\text{mod } b \ a = 0$ if $a = 0$; $\text{op } b(\text{mod } b(a \text{ div } 2), 0)$ if a even;
 $\text{op } b(\text{mod } b(a \text{ div } 2), 1)$ if a odd — $O(\log a)$.

6.5 Unique fixed points

A hylomorphism is the *least* fixed point of its recursion, not necessarily the only one.

- 6.5 On Nat take $X = ([\text{zero}, \text{positive}])^\circ ([\text{zero}, 1])$ with $\text{positive} = \text{succ}^\circ \text{succ}$ the coreflexive on positives. Both catamorphisms describe the coreflexive on 0, so X is that. But its recursion $X = (\text{zero}^\circ \text{zero}) \cup (\text{positive } X)$ also has $X = 1$ as a solution.

$[\text{zero}, \text{positive}]^\circ$ and $[\text{zero}, 1]$ are both maps, yet the least solution is not even entire. Simplicity does carry over (Ex 6.10); entirety is what needs a hypothesis, and the hypothesis is that a membership relation be *inductive*.

Inductive and well-founded relations

- 6.5 $\checkmark$ $R : \alpha \rightarrow \alpha$ is **INDUCTIVE** $\triangleq$ for all X ,

$$X/R \subset X \implies \Pi \subset X.$$

Pointwise: if $(\forall c. cRa \implies cXb) \implies aXb$ holds for all a, b , then X holds everywhere — exactly a proof by induction over R .

$<$ on Nat gives ordinary mathematical induction; $\text{tail} = \text{cons}^\circ \pi'$ gives induction over lists.

- Ex 6.13 $\checkmark$ S inductive and $RR \subset SR \implies R$ inductive. Hence $R \subset S$ with S inductive $\implies R$ inductive, and S inductive $\iff S^+$ inductive, where $S^+ = (\mu X : S \cup (SX))$.

- Ex 6.15 $\checkmark$ The meet of two inductive relations is inductive; the join need not be.

- 6.5 $\checkmark$ R is **WELL-FOUNDED** $\triangleq$ for all X , $X \subset RX \implies X = \emptyset$ — no infinite chain $a_0 Ra_1 R \dots$. Inductive $\implies$ well-founded, and the converse holds in a Boolean allegory.

- Ex 6.16 $\checkmark$ R well-founded $\implies fRf^\circ$ well-founded, for any map f .

Membership

- 6.5 A relator F may carry a **MEMBERSHIP** arrow $\text{mem}(F)_\alpha : F\alpha \rightarrow \alpha$ with $a \text{ mem } x$ exactly when a is an element of x . For the polynomial relators, the power relator and the type relators it exists:

$\text{mem}(\text{Id}) = 1$, $\text{mem}(K_\beta) = \emptyset$, $\text{mem}(F + G) = [\text{mem}(F), \text{mem}(G)]$.

$\text{mem}(F \times G) = (\pi \text{ mem}(F)) \cup (\pi' \text{ mem}(G))$, $\text{mem}(F \circ G) = \text{mem}(F) \text{ mem}(G)$, $\text{mem}([\cdot]) = \exists$.

Type relators, via sets: $\text{mem}(T) = \text{setify}(T) \ni$ with $\text{setify}(T) = \Lambda(\text{mem}(T))$ — Ex 6.17 gives a set-free alternative.

mem is a lax natural transformation $1 \hookrightarrow F$, that is $(FR) \text{ mem} \subset \text{mem} R$, and it is the *largest* one of that type — so a membership relation, if it exists, is unique.

6.5 ✓ The result the section is for: $\text{in}^\circ \text{ mem}(F)$ is inductive. It is the immediate-subterm relation.

Nat, $FX = 1 + X$: $\text{mem} = [\emptyset, 1]$ and $[\text{zero}, \text{succ}]^\circ[\emptyset, 1] = \text{succ}^\circ$ is inductive; $< = \text{pred}^+$ with $\text{pred} = \text{succ}^\circ$, which is what made §6.1's recursion terminate.

Lists: $\text{mem} = [\emptyset, \pi']$ and $[\text{nil}, \text{cons}]^\circ[\emptyset, \pi'] = \text{cons}^\circ \pi' = \text{tail}$ is inductive, so proper suffix tail^+ is; on snoc -lists init and proper prefix are.

Unique solutions

T 6.3 ✓ $S \text{ mem}(F) \text{ inductive} \implies X = S(FX)R$ has a unique solution $\varphi(R, S)$, and $\varphi(R, S)$ is entire if R and S are. With $S := S^\circ$ this is the hylomorphism of §6.3, now pinned down as the *only* fixed point.

C 6.2 ✓ $g \text{ mem}(F) \text{ inductive} \implies$ the unique solution of $X = g(FX)f$ is a map $(X = \langle g \rangle^\circ \langle f \rangle)$ is entire by the theorem and simple by Ex 6.10).

C 6.3 ✓ $R^\circ \text{ mem}(F) \text{ inductive} \implies \langle R \rangle$ is surjective when R is (R surjective $\triangleq 1 \subset R^\circ R$, i.e. $\text{ran } R = 1$).

T 6.4 ✓ R surjective and $Rf \subset (Ff) \text{ in} \implies f^\circ = \langle R \rangle$. This is the theorem §5.6 borrowed to show $\text{partition} = \text{concat}^\circ$ is a catamorphism.

Ex 6.12 ✓ R inductive $\iff X = X/R$ has a unique solution.

Ex 6.17 $\text{inlist} : \text{list}^+ \alpha \rightarrow \alpha$ is a catamorphism, but inlist on $\text{list } \alpha$ is not; instead $\text{inlist} = \text{tail}^* \text{ head}$. The same trick defines intree .

Ex 6.18 The formal definition: mem is a membership relation for F if $(1/\text{ mem})(FR) = R/\text{ mem}$ for all

R ; equivalently $(S/\text{mem})(FR) = (SR)/\text{mem}$ for all R, S .

Ex 6.20 $\text{mem}(G)/\text{mem}(F)$ is the largest lax natural transformation $F \hookrightarrow G$.

Ex 6.21 In a Boolean allegory, $\text{mem}(F)$ is entire $\iff F\emptyset = \emptyset$.

6.6 Sorting by selection

(6.6) ✓ $\text{sort} \subset \text{perm}$ (ordered), for R a *connected* preorder ($R \cup R^\circ = \Pi$). A connected preorder need not be anti-symmetric, so sort is genuinely a refinement, not an equality.

ordered = $\langle \text{nil}, \text{ok cons} \rangle$, where the coreflexive ok holds at (a, x) when $(\forall b : b \text{ inlist } x : aRb)$ — rebuild the list, checking at each step that only R -smaller elements go in front. ($\text{ok}(a, x) = (x = [] \vee aR(\text{head } x))$ also works but is less useful here.)

Selection sort

$$\begin{aligned}
 & \text{perm (ordered)} \\
 = & \text{perm} = \text{perm}^\circ \text{ and ordered} = \text{ordered}^\circ \\
 & (\text{ordered perm})^\circ \\
 = & \text{ordered} = \langle \text{nil}, \text{ok cons} \rangle \\
 & (\langle \text{nil}, \text{ok cons} \rangle \text{ perm})^\circ \\
 \supseteq & \text{fusion, for a suitable select} \\
 & \langle \text{nil}, \text{select}^\circ \rangle
 \end{aligned}$$

6.6 The fusion proviso is $\text{ok cons perm} \supset (1 \times \text{perm})\text{select}^\circ$, and select is found by calculating:

$$\begin{aligned}
 & \text{ok cons perm} \\
 = & \text{perm} = \langle \text{nil}, \text{cons perm} \rangle \text{ (§5.6)} \\
 & \text{ok } (1 \times \text{perm}) \text{ cons perm} \\
 = & (1 \times \text{perm})\text{ok} = \text{ok}(1 \times \text{perm}), \text{ Ex 6.22} \\
 & (1 \times \text{perm}) \text{ ok cons perm} \\
 \supseteq & \text{specifying } \text{select} \subset \text{perm cons}^\circ \text{ ok} \\
 & (1 \times \text{perm})\text{select}^\circ
 \end{aligned}$$

6.6 In words: $(a, y) = \text{select } x$ means $[a] \# y$ is a permutation of x with aRb for every b in y . It is undefined on $[]$, so take $\text{select} = \text{embed } \langle \text{base}, \text{step} \rangle$ with $\text{embed} : \text{list } \alpha \rightarrow \text{list}^+ \alpha$.

Fusion conditions: $\text{base} \subset \text{wrap perm cons}^\circ \text{ ok}$ and $(1 \times (\text{cons}^\circ \text{ ok})) \text{ step} \subset \text{cons perm cons}^\circ \text{ ok}$, satisfied by $\text{base } a = (a, [])$ and $\text{step}(a, (b, x)) = (a, [b] \# x)$ if aRb , else $(b, [a] \# x)$.

6.6 ✓ The hyломorphism theorem then makes $X = \langle \text{nil}, \text{select}^\circ \rangle$ the unique solution of $X = (\text{nil}^\circ \text{ nil}) \cup$

(select $(1 \times X)$ cons), i.e. sort $x = []$ if $x = []$, else
 $[a] \# \text{sort } y$ where $(a, y) = \text{select } x$.

Quicksort

6.6 Same shape, with a tree as the intermediate datatype:
 tree $\alpha ::= \text{null} \mid \text{fork}(\text{tree } \alpha, \alpha, \text{tree } \alpha)$ and flatten =
 $(\text{nil}, \text{join})$, $\text{join}(x, a, y) = x \# [a] \# y$.

perm (ordered)
 $\supseteq$ flatten a map, so $\text{flatten}^\circ \text{flatten} \subset 1$
 perm $\text{flatten}^\circ \text{flatten}$ ordered
 = claim: $\text{flatten ordered} = \text{inordered flatten}$
 perm $\text{flatten}^\circ \text{inordered flatten}$
 = converses
 $(\text{inordered flatten perm})^\circ \text{flatten}$
 $\supseteq$ fusion, for a suitable split
 $(\text{nil}, \text{split}^\circ) \text{flatten}$

6.6 $\text{inordered} = (\text{nil}, \text{check fork})$, with check holding at
 (x, a, y) when $(\forall b : b \text{intree } x \implies bRa)$ and $(\forall b : b \text{intree } y \implies aRb)$; check' is the same with inlist for
 intree. Writing $Ff = f \times 1 \times f$, the proviso is
 $F(\text{flatten perm})\text{split}^\circ \subset \text{check fork flatten perm}$.

Discharged by taking $\text{split} \subset \text{perm join}^\circ \text{check}'$, i.e.
 $(y, a, z) = \text{split } x$ means $y \# [a] \# z$ is a permutation
 of x with bRa throughout y and aRb throughout z .

$\text{split} = \text{embed } (\text{base}, \text{step})$ with $\text{base} \subset$
 $\text{wrap perm join}^\circ \text{check}'$ and $(1 \times (\text{join}^\circ \text{check}')) \text{step} \subset$
 $\text{cons perm join}^\circ \text{check}'$ (the book prints split for step
 here, and drops the $^\circ$ on join), satisfied by base $a =$
 $([], a, [])$ and $\text{step}(a, (x, b, y)) = ([a] \# x, b, y)$ if aRb ,
 else $(x, b, [a] \# y)$.

6.6 $X = (\text{nil}, \text{split}^\circ) \text{flatten}$ is the least solution of $X =$
 $(\text{nil}^\circ \text{nil}) \cup (\text{split}(X \times 1 \times X)\text{join})$, i.e. sort $x = []$ if
 $x = []$, else sort $y \# [a] \# \text{sort } z$ where $(y, a, z) =$
 $\text{split } x$.

Ex 6.22 $\text{inlist} = \text{perm inlist}$; hence a point-free ok and $(1 \times$
 $\text{perm})\text{ok} = \text{ok}(1 \times \text{perm})$.

Ex 6.24 $\text{ordered}(R) \text{ordered}(S) = \text{ordered}(R \cap S)$.

Ex 6.25 Sorting a set: $\text{sort} \subset \text{setify}^\circ \text{ordered}(R)$ is the wrong
 specification, $\text{sort} \subset \text{setify}^\circ \text{ordered}(R \cap \text{neq})$ the
 right one, where $a \text{neq } b$ iff $a \neq b$.

Ex 6.30 From $\text{perm} = \langle \text{nil}, \text{add} \rangle$ instead, fusion gives
 $\text{perm}(\text{ordered}) = \langle \text{nil}, \text{add ordered} \rangle \supset \langle \text{nil}, \text{insert} \rangle$ for
a suitable map insert with $(1 \times \text{ordered})\text{insert} \subset$
 add ordered — insertion sort.

6.7 Closure

6.7 ✓ R^* , the **REFLEXIVE TRANSITIVE CLOSURE**, is the smallest preorder containing R :

$$R \subset X \iff R^* \subset X \quad \text{for every preorder } X.$$

(6.7) ✓ $R^* = (\mu X : 1 \cup (RX)).$

(6.8) ✓ $R^* = (\mu X : 1 \cup (XR)).$ That $S = (\mu X : 1 \cup (RX))$ is reflexive and contains R is immediate; transitivity goes through division, the same move as Theorem 6.2.

Ex 6.32 ✓ $SR^* = (\mu X : S \cup (XR)), \quad RS^* = (\mu X : R \cup (XS)), \quad S^*R = (\mu X : R \cup (SX)).$

Ex 6.33 $R^* = \llbracket [1, 1] \rrbracket^\circ \llbracket [1, R] \rrbracket$, the intermediate datatype being $\text{iterate } \alpha ::= \text{once } \alpha \mid \text{again}(\text{iterate } \alpha).$

Ex 6.34 $R^* = \text{chainR}^\circ$ head for a catamorphism chainR on non-empty lists.

Ex 6.36 $(R \cup S)^* = (R^*S)^*R^*$, by the diagonal rule.

Ex 6.37 $CR = C \implies R^* = (\sim C R)^*$, for C coreflexive.

When the recursion has a unique solution

6.7 $X = 1 \cup (RX)$ has a unique solution — necessarily R^* — exactly when R is inductive. tail is inductive, so $\text{suffix} = 1 \cup (\text{tail suffix})$ pins suffix down, and transposing gives $\Lambda \text{ suffix} = \langle \tau, \Lambda(\text{tail suffix}) \rangle \text{ cup}.$

tail is undefined on $\llbracket \rrbracket$, so a case analysis splits it:

$(\Lambda \text{ suffix})\llbracket \rrbracket = \{\llbracket \rrbracket\}$ and $(\Lambda \text{ suffix})([a] \# x) = \{[a] \# x\} \cup (\Lambda \text{ suffix})x$ — sets as lists, and this is §5.6's $\text{tails } \llbracket \rrbracket = \{\llbracket \rrbracket\}$, $\text{tails}([a] \# x) = [[a] \# x] \# \text{tails } x.$

Computing a closure that is not a unique fixed point

6.7 **SUBTRACTION** (Ex 4.30) $R - S \subset T \iff R \subset S \cup T$; hence $R - \emptyset = R$, $R \cup S = R \cup (S - R)$, $R - (S \cup T) = R - S - T$ and $(R \cup S) - T = (R - T) \cup (S - T).$

Ex 6.35 ✓ **M-CALCULUS** the two defining rules are $\varphi(\mu X : \varphi X) = (\mu X : \varphi X)$ and $\varphi Y \subset Y \implies (\mu X : \varphi X) \subset Y$. From them: *rolling* $(\mu X : \varphi(\psi X)) = \varphi(\mu X : \psi(\varphi X))$, *diagonal* $(\mu X : \mu Y : \varphi(X, Y)) = (\mu X :$

$\varphi(X, X)$, *substitution* $(\mu X : \varphi(\mu Y : \psi(X, Y))) = (\mu X : \psi(\varphi X, X))$.

(6.9) $\theta(P, Q) \triangleq P \cup (\mu X : Q \cup ((XR) - P))$, which satisfies

$$\begin{aligned}\theta(\emptyset, S) &= SR^*, \quad \theta(P, \emptyset) = P, \\ \theta(P, Q) &= \theta(P \cup Q, (QR) - P - Q).\end{aligned}$$

The third comes out of the rolling rule and the subtraction laws.

Read as a program: compute $\theta(\emptyset, S)$, where $\theta(P, Q) = P$ if $Q = \emptyset$, else $\theta(P \cup Q, (QR) - P - Q)$. It terminates only if SR^* is finite; then P grows at every step.

Relations as data are awkward, so apply Λ throughout: with $p = \Lambda P$, $q = \Lambda Q$, $s = \Lambda S$ and $\Lambda(SR^*) = (\Lambda S)(ER^*)$,

$$\text{close}(p, q) = \begin{cases} p & \text{if } q = \emptyset \\ \text{close}(p \cup q, (ER)q - p - q) & \text{otherwise,} \end{cases}$$

now with set union and set difference. This is how $E(R^*)$ is computed whenever the answer is known to be finite — reachability in a graph.